PART 5 · TERRAFORM STUDY NOTES

# Terraform Variables, Locals & Outputs

Professional Study Notes · Beginner to Advanced

English Notes + Simple Spoken Kannada Explanations · AWS, Jenkins & EKS examples

ಈ notes ನಲ್ಲಿ Terraformನ Input Variables, Variable Types, Defaults, Validation, .tfvars, Locals, Outputs ಮತ್ತು Sensitive Values ಎಲ್ಲವನ್ನೂ practical AWS, Jenkins ಮತ್ತು EKS project examples ಜೊತೆಗೆ ಕಲಿಯೋಣ.

ಪ್ರತಿ topicಗೂ ನೀವು ಕೇಳಿದ ಐದು headings ಇರುತ್ತವೆ:

1. Definition — What is it?
2. Purpose — Why do we use it?
3. Working — How does it work?
4. Practical Example — How do we use it in a project?
5. Scenario-Based Interview Questions & Answers — How can interviewers ask?

## 1. Input Variables in Terraform

### 1. Definition — What is an input variable?

An input variable is a parameter that allows us to pass values into Terraform configurations without hardcoding those values in resource blocks.

Input variables are generally declared in a **variables.tf** file and referenced using **var.variable_name**.

> **ಕನ್ನಡ ವಿವರಣೆ:** Input variable ಅಂದ್ರೆ ನಮ್ಮ Terraform code ಒಳಗೆ values ಅನ್ನು ಹೊರಗಿನಿಂದ ಕೊಡುವ ವ್ಯವಸ್ಥೆ. ಒಂದೇ code ಅನ್ನು ಮತ್ತೆ ಮತ್ತೆ ಬದಲಾಯಿಸದೇ ಬೇರೆ ಬೇರೆ values ಬಳಸಿ resources create ಮಾಡಬಹುದು.

### 2. Purpose — Why do we use input variables?

- To avoid hardcoding values.
- To reuse the same Terraform configuration across environments.
- To make configuration easier to maintain.
- To pass values from Jenkins pipelines or .tfvars files.
- To separate configuration values from infrastructure logic.

> **ಕನ್ನಡ ವಿವರಣೆ:** ಉದಾಹರಣೆಗೆ Dev, Test, Production environmentsಗೆ ಬೇರೆ ಬೇರೆ AWS regions ಅಥವಾ instance types ಬೇಕಾಗಬಹುದು. ಪ್ರತಿ ಸಲ main.tf ಬದಲಾಯಿಸುವ ಬದಲು variables ಮೂಲಕ values ಕೊಡಬಹುದು.

### 3. Working — How does it work?

1. Declare a variable in variables.tf.

2. Assign a value through a default, .tfvars file, CLI argument, environment variable, or interactive prompt.
3. Reference the variable in Terraform configuration using var.name.
4. Terraform evaluates the value while creating the execution plan.

## 4. Practical Example — AWS EC2

**variables.tf**

```
variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t3.micro"
}
```

**main.tf**

```
resource "aws_instance" "app" {
  ami           = var.ami_id
  instance_type = var.instance_type
}
```

The **ami_id** variable must also be declared and assigned a valid AMI ID for the selected AWS region.

Override the instance type from the command line:

```
terraform plan -var="instance_type=t3.small"
```

> **ಕನ್ನಡ ವಿವರಣೆ:** ಇಲ್ಲಿ default ಆಗಿ t3.micro ಇದೆ. ಆದರೆ command ಮೂಲಕ t3.small ಕೊಟ್ಟರೆ, ಆ Terraform runನಲ್ಲಿ ಅದನ್ನೇ ಬಳಸುತ್ತದೆ. ಇದರಿಂದ code ಬದಲಾಯಿಸದೇ instance type ಬದಲಾಯಿಸಬಹುದು.

## 5. Scenario-Based Interview Questions & Answers

**Q1. Why do you use variables in Terraform?**

**Answer:** I use variables to avoid hardcoding, improve reusability, and configure different environments without changing the Terraform resource definitions.

**Q2. How do you pass a variable value from the command line?**

**Answer:**

```
terraform plan -var="instance_type=t3.small"
```

**Q3. What happens if a variable has no default value and I do not provide one?**

**Answer:** Terraform normally prompts for the missing value in interactive mode. In non-interactive execution, such as a Jenkins pipeline, the run can fail if the required value is not supplied.

# 2. Variable Types in Terraform

## 1. Definition — What are variable types?

Variable types specify what kind of data a Terraform input variable accepts. Common types include:

- **string** — text
- **number** — integer or decimal number
- **bool** — true or false
- **list(string)** — ordered collection of strings
- **set(string)** — unique collection of strings
- **map(string)** — key-value pairs with string values
- **object({...})** — structured data with named attributes
- **tuple([...])** — ordered values whose types are defined by position

## 2. Purpose — Why do we use variable types?

- To keep input values consistent.
- To catch invalid configuration early.
- To make Terraform code easier to understand.
- To model complex infrastructure settings.
- To improve maintainability in large AWS and EKS projects.

> **ಕನ್ನಡ ವಿವರಣೆ:** Variableಗೆ ಯಾವ ರೀತಿಯ value ಕೊಡಬೇಕು ಅಂತ type ನಿರ್ಧರಿಸುತ್ತದೆ. ಉದಾಹರಣೆಗೆ number variableಗೆ ಸಂಖ್ಯೆ ಕೊಡಬೇಕು; bool variableಗೆ true ಅಥವಾ false ಕೊಡಬೇಕು. ತಪ್ಪಾದ value ಕೊಟ್ಟರೆ Terraform error ತೋರಿಸಬಹುದು.

## 3. Working — How do variable types work?

Terraform checks the supplied value against the declared type and may convert compatible values where possible. If a value cannot satisfy the required type, Terraform reports an error.

## 4. Practical Example — Different types

```
variable "project_name" {
  type    = string
  default = "EKS-Project"
}

variable "desired_capacity" {
  type    = number
  default = 2
}

variable "enable_monitoring" {
  type    = bool
  default = true
}

variable "availability_zones" {
  type    = list(string)
  default = ["us-east-1a", "us-east-1b"]
}

variable "common_tags" {
  type = map(string)

  default = {
    Environment = "Dev"
    ManagedBy   = "Terraform"
  }
}
```

Reference these values:

```
var.project_name
var.desired_capacity
var.enable_monitoring
var.availability_zones
var.common_tags
```

> **ಕನ್ನಡ ವಿವರಣೆ:** EKS clusterಗೆ ಎಷ್ಟು worker nodes ಬೇಕು ಅನ್ನೋದಕ್ಕೆ number, monitoring ON/OFF ಮಾಡಲು bool, availability zones ಪಟ್ಟಿ ಮಾಡಲು list(string) ಬಳಸಬಹುದು.

## 5. Scenario-Based Interview Questions & Answers

**Q1. What is the difference between a list and a set?**

**Answer:** A list is ordered and can contain duplicate values. A set contains unique values and does not guarantee positional ordering.

**Q2. When would you use a map?**

**Answer:** I use a map when I need related key-value settings, such as environment tags or configuration values.

**Q3. What is an object type?**

**Answer:** An object defines a structured value with named attributes and specified types. It is useful for grouping related configuration settings.

# 3. Default Values in Terraform

## 1. Definition — What is a default value?

A default value is a predefined value assigned to an input variable. Terraform uses it when no higher-precedence value is provided for that variable.

## 2. Purpose — Why do we use defaults?

- To reduce repetitive input.
- To provide standard configuration values.
- To make simple Terraform projects easier to run.
- To define sensible development environment settings.

**ಕನ್ನಡ ವಿವರಣೆ:** Variableಗೆ ಮೊದಲೇ ಒಂದು value ಇಟ್ಟಿದ್ದರೆ, ಪ್ರತಿ ಸಲ ಅದನ್ನು ನಾವು ಕೊಡಬೇಕಾಗಿಲ್ಲ. ಬೇರೆ value ಕೊಡದಿದ್ದರೆ Terraform ಆ default value ಬಳಸುತ್ತದೆ.

## 3. Working — How does it work?

If a variable has a default, Terraform uses that default when no other value is supplied. If you explicitly provide a value through a supported input mechanism, Terraform uses the supplied value according to variable precedence.

## 4. Practical Example

```
variable "aws_region" {
  description = "AWS deployment region"
  type        = string
  default     = "us-east-1"
}

variable "instance_type" {
  type    = string
  default = "t3.micro"
}
```

Override the default:

```
terraform plan -var="aws_region=us-east-2"
```

**ಕನ್ನಡ ವಿವರಣೆ:** Default region us-east-1. ಆದರೆ commandನಲ್ಲಿ us-east-2 ಕೊಟ್ಟಿದ್ದರಿಂದ ಈ runಗೆ ಆ region ಬಳಸಲಾಗುತ್ತದೆ.

### 5. Scenario-Based Interview Questions & Answers

**Q1. Can a variable have no default value?**

**Answer:** Yes. A variable without a default is required unless its value is supplied through another supported input mechanism.

**Q2. Should we always set defaults for production variables?**

**Answer:** No. Defaults should be used when they are safe and appropriate. Important production settings may be better supplied explicitly to avoid accidental deployments with unintended values.

**Q3. What happens if I override a default?**

**Answer:** Terraform uses the supplied value instead of the default for that run.

## 4. Variable Validation

### 1. Definition — What is variable validation?

Variable validation is a mechanism that checks whether an input value satisfies specified conditions before Terraform proceeds with the operation.

### 2. Purpose — Why do we use validation?

- To prevent invalid configuration values.
- To enforce naming rules.
- To restrict instance types or numeric ranges.
- To catch mistakes before resource creation.
- To improve the reliability of CI/CD pipelines.

**ಕನ್ನಡ ವಿವರಣೆ:** User ತಪ್ಪಾದ value ಕೊಟ್ಟರೆ Terraform ಮುಂದಕ್ಕೆ ಹೋಗುವ ಮೊದಲು error ತೋರಿಸಬಹುದು. ಉದಾಹರಣೆಗೆ instance count 1ಕ್ಕಿಂತ ಕಡಿಮೆ ಇರಬಾರದು ಅಂತ rule ಹಾಕಬಹುದು.

### 3. Working — How does it work?

A variable's **validation** block contains a Boolean **condition** and an explanatory **error_message**. If the condition evaluates to false, Terraform reports a validation error.

## 4. Practical Example

```
variable "environment" {
  description = "Deployment environment"
  type        = string

  validation {
    condition = contains(
      ["dev", "test", "prod"],
      var.environment
    )

    error_message = "Environment must be dev, test, or prod."
  }
}
```

Another example for EKS node capacity:

```
variable "desired_capacity" {
  type    = number
  default = 2

  validation {
    condition = (
      var.desired_capacity >= 1 &&
      var.desired_capacity <= 10
    )

    error_message = "Desired capacity must be between 1 and 10."
  }
}
```

**ಕನ್ನಡ ವಿವರಣೆ:** environmentಗೆ dev, test, prod ಮಾತ್ರ ಕೊಡಬಹುದು. production ಅಂತ ಬೇರೆ value ಕೊಟ್ಟರೆ validation fail ಆಗುತ್ತದೆ. ಇದರಿಂದ ತಪ್ಪಾದ configuration deploymentಗೆ ಹೋಗುವುದನ್ನು ತಡೆಯಬಹುದು.

## 5. Scenario-Based Interview Questions & Answers

**Q1. What is the difference between variable type checking and validation?**

**Answer:** Type checking verifies the kind and structure of a value. Validation checks additional business rules, such as allowed environment names or numeric limits.

**Q2. How would you prevent an EKS node count from exceeding a limit?**

**Answer:** I would define a numeric input variable and add a validation condition that enforces the permitted range.

**Q3. Does validation replace AWS permissions and security controls?**

**Answer:** No. Validation checks configuration inputs. AWS IAM, security groups, policies, and other controls are still required.

# 5. .tfvars Files

## 1. Definition — What is a .tfvars file?

A **.tfvars** or **.tfvars.json** file supplies values for Terraform input variables separately from their declarations.

## 2. Purpose — Why do we use .tfvars?

- To separate environment-specific values from Terraform code.
- To reuse the same configuration for Dev, Test, and Production.
- To simplify CI/CD execution.
- To organize many input values in one place.

> **ಕನ್ನಡ ವಿವರಣೆ:** variables.tfನಲ್ಲಿ variable ಹೆಸರು ಮತ್ತು type ಇರುತ್ತದೆ. .tfvarsನಲ್ಲಿ ಆ variableಗೆ ಯಾವ value ಕೊಡಬೇಕು ಅಂತ ಬರೆಯುತ್ತೇವೆ.

## 3. Working — How does it work?

Create a values file and pass it to Terraform with **-var-file**. Terraform also automatically loads certain conventional filenames, including **terraform.tfvars** and ***.auto.tfvars**, when present in the root module.

## 4. Practical Example

**variables.tf**

```
variable "aws_region" {
  type = string
}

variable "instance_type" {
  type = string
}
```

**dev.tfvars**

```
aws_region   = "us-east-1"
instance_type = "t3.micro"
```

**prod.tfvars**

```
aws_region   = "us-east-2"
instance_type = "t3.small"
```

Run the development configuration:

```
terraform plan -var-file="dev.tfvars"
```

Run the production configuration:

```
terraform plan -var-file="prod.tfvars"
```

**ಕನ್ನಡ ವಿವರಣೆ:** ಒಂದೇ Terraform code ಬಳಸಿ Devಗೆ t3.micro, Productionಗೆ t3.small ಬಳಸಬಹುದು. ಯಾವ .tfvars file ಆಯ್ಕೆ ಮಾಡುತ್ತೇವೋ ಅದರ values ಆ runಗೆ ಅನ್ವಯಿಸುತ್ತವೆ.

### 5. Scenario-Based Interview Questions & Answers

**Q1. What is the difference between variables.tf and terraform.tfvars?**

**Answer:** variables.tf declares input variables and their types, defaults, and validation rules. terraform.tfvars supplies values for those variables.

**Q2. How do you use separate configuration for Dev and Production?**

**Answer:** I maintain separate variable files and run Terraform with the appropriate -var-file option.

**Q3. Should we commit secrets in .tfvars to GitHub?**

**Answer:** No. Sensitive values should not be committed to source control. Use an approved secrets manager or secure CI/CD credential mechanism, and restrict access to state files and plan artifacts.

## 6. Locals in Terraform

### 1. Definition — What are local values?

Local values are named expressions defined in a **locals** block. They help assign a name to a value or expression that can be reused throughout the current module.

### 2. Purpose — Why do we use locals?

- To avoid repeating expressions.
- To create common resource tags.
- To combine input variables.
- To simplify complex expressions.
- To improve readability and maintainability.

**ಕನ್ನಡ ವಿವರಣೆ:** ಒಂದೇ expression ಅಥವಾ value ಅನ್ನು ಹಲವಾರು ಕಡೆ ಬಳಸಬೇಕಾದರೆ ಅದನ್ನು localsನಲ್ಲಿ ಒಮ್ಮೆ define ಮಾಡಿ, ಬೇರೆ ಬೇರೆ resourcesನಲ್ಲಿ ಬಳಸಬಹುದು.

### 3. Working — How do locals work?

Terraform evaluates local expressions from their referenced inputs and other expressions. You reference a local value using **local.name**, not **var.name**.

### 4. Practical Example — Common AWS tags

**locals.tf**

```
locals {
  common_tags = {
    Project     = "EKS-Project"
    Environment = var.environment
    ManagedBy   = "Terraform"
  }

  name_prefix = "${var.project_name}-${var.environment}"
}
```

**main.tf**

```
resource "aws_instance" "app" {
  ami           = var.ami_id
  instance_type = var.instance_type

  tags = merge(local.common_tags, {
    Name = "${local.name_prefix}-app"
  })
}
```

> **ಕನ್ನಡ ವಿವರಣೆ:** ಎಲ್ಲಾ AWS resourcesಗೆ ಒಂದೇ ರೀತಿಯ tags ಬೇಕಾದರೆ common_tags ಬಳಸಬಹುದು. Environment dev ಆಗಿದ್ದರೆ resource tagsನಲ್ಲಿ ಅದೇ value ಬರುತ್ತದೆ. ಮುಂದೆ ಬದಲಾವಣೆ ಮಾಡಬೇಕಾದರೆ ಒಂದೇ ಸ್ಥಳದಲ್ಲಿ ಬದಲಾಯಿಸಬಹುದು.

### 5. Scenario-Based Interview Questions & Answers

**Q1. What is the difference between variables and locals?**

**Answer:** Variables receive inputs from users, files, environment variables, or automation. Locals define reusable expressions calculated within the module.

**Q2. Can a local value be overridden using -var?**

**Answer:** No. A local value is an expression, not a direct input variable. You can change the inputs or expression that determine it.

**Q3. Why are locals useful in large Terraform projects?**

**Answer:** They reduce duplication, centralize common expressions and tags, and make configurations easier to maintain.

## 7. Outputs in Terraform

### 1. Definition — What are output values?

Output values expose selected information from a Terraform module after evaluation. They can display useful resource attributes and pass values from child modules to parent modules.

### 2. Purpose — Why do we use outputs?

- To display an EC2 public IP or DNS name.

- To retrieve an EKS cluster name.
- To expose a VPC ID or load balancer endpoint.
- To pass values between Terraform modules.
- To provide useful information to deployment automation.

> **ಕನ್ನಡ ವಿವರಣೆ:** Terraform resource create ಮಾಡಿದ ನಂತರ ಅದರ IP address, DNS name ಅಥವಾ cluster name ಬೇಕಾಗಬಹುದು. ಆ details ಅನ್ನು ಸುಲಭವಾಗಿ ನೋಡಲು outputs.tf ಬಳಸುತ್ತೇವೆ.

## 3. Working — How do outputs work?

An **output** block defines a name and a value expression. After an apply, Terraform displays root-module outputs. You can also retrieve a specific output with **terraform output**.

## 4. Practical Example — EC2 output

**outputs.tf**

```
output "instance_public_ip" {
  description = "Public IP of the EC2 instance"
  value       = aws_instance.app.public_ip
}

output "instance_public_dns" {
  description = "Public DNS of the EC2 instance"
  value       = aws_instance.app.public_dns
}
```

Commands:

```
terraform apply
terraform output
terraform output instance_public_ip
```

For an EKS project:

```
output "eks_cluster_name" {
  description = "EKS cluster name"
  value       = aws_eks_cluster.eks.name
}
```

> Note: The resource addresses above are examples. Use the actual resource names from your configuration.

> **ಕನ್ನಡ ವಿವರಣೆ:** EC2 create ಆದ ನಂತರ IP address ನೋಡಲು AWS Consoleಗೆ ಪ್ರತಿಸಾರಿ ಹೋಗಬೇಕಾಗಿಲ್ಲ. terraform output instance_public_ip command ಮೂಲಕ ನೋಡಬಹುದು.

## 5. Scenario-Based Interview Questions & Answers

**Q1. What is the difference between a resource and an output?**

**Answer:** A resource declares infrastructure Terraform manages. An output exposes a value from that infrastructure or another expression.

**Q2. Can a child module's output be accessed directly from the root module?**

**Answer:** The child module must declare an output, and the root module references it using module.<module_name>.<output_name>.

**Q3. Can outputs contain sensitive information?**

**Answer:** Yes. Outputs can contain sensitive values, but they must be explicitly marked sensitive when appropriate. Marking an output sensitive hides it in some CLI displays; it does not guarantee that the value is absent from state.

# 8. Sensitive Values in Terraform

## 1. Definition — What are sensitive values?

Sensitive values are input variables or outputs that Terraform should treat as confidential in its user-facing output. Examples include passwords, tokens, and secret strings.

## 2. Purpose — Why do we use them?

- To reduce accidental exposure in terminal output.
- To avoid displaying secrets during routine operations.
- To make the handling of confidential values more deliberate.
- To improve the safety of automated workflows.

**ಕನ್ನಡ ವಿವರಣೆ:** Password ಅಥವಾ token ಅನ್ನು terminal outputನಲ್ಲಿ ಎಲ್ಲರಿಗೂ ಕಾಣುವಂತೆ ತೋರಿಸಬಾರದು. sensitive = true ಕೊಟ್ಟರೆ Terraform ಸಾಮಾನ್ಯ outputಗಳಲ್ಲಿ ಆ value ಅನ್ನು ಮರೆಮಾಡುತ್ತದೆ. ಆದರೆ ಇದು ಸಂಪೂರ್ಣ secret storage solution ಅಲ್ಲ.

## 3. Working — How does it work?

Terraform supports **sensitive = true** on variable and output blocks. This marks the value as sensitive and redacts it in many CLI outputs.

Important: Sensitive values can still be stored in Terraform state and plan files. Protect those files with appropriate access controls and encryption. Avoid placing secrets directly in command-line arguments or source control.

## 4. Practical Example

```
variable "db_password" {
  description = "Database password"
  type        = string
  sensitive   = true
}
```

A sensitive output:

```
output "db_password" {
  value     = var.db_password
  sensitive = true
}
```

For real projects, prefer an appropriate secrets-management solution, such as AWS Secrets Manager, with access controlled through IAM. Do not hardcode actual passwords in these examples.

> **ಕನ್ನಡ ವಿವರಣೆ:** sensitive = true ಮಾಡಿದರೆ Terraform outputನಲ್ಲಿ password ಸಾಮಾನ್ಯವಾಗಿ ಮರೆಮಾಡಲ್ಪಡುತ್ತದೆ. ಆದರೆ state fileನಲ್ಲಿ secret ಉಳಿಯುವ ಸಾಧ್ಯತೆ ಇದೆ. ಆದ್ದರಿಂದ state fileಗೆ secure storage ಮತ್ತು access control ಬಹಳ ಮುಖ್ಯ.

### 5. Scenario-Based Interview Questions & Answers

**Q1. Does sensitive = true encrypt a secret in the Terraform state file?**

**Answer:** No. It primarily controls how Terraform displays sensitive values. State encryption and access protection must be handled separately.

**Q2. How would you pass a secret securely in Jenkins?**

**Answer:** I would store the secret in Jenkins Credentials or an approved secrets manager, restrict access to the credential, and inject it into the pipeline only when needed. I would also avoid printing it in logs.

**Q3. Is a sensitive output completely hidden from everyone?**

**Answer:** No. Users with access to the underlying state or other ways of retrieving the value may still be able to access it. Sensitive marking is not a substitute for access control.

## 9. Advanced Interview Revision

| Topic | Key difference |
|---|---|
| Variables vs. Locals | Inputs vs. reusable expressions |
| Variables vs. Outputs | Receive values vs. expose values |
| variables.tf vs. .tfvars | Declarations vs. assigned values |
| Default vs. Validation | Fallback value vs. rule checking |
| sensitive = true vs. encryption | Display redaction vs. protection of stored data |

### Important Terraform commands

```
terraform fmt
terraform validate
terraform plan
terraform plan -var-file="dev.tfvars"
terraform apply
terraform output
terraform output instance_public_ip
```

## 10. Real-Time Project Scenario — Jenkins + AWS + EKS

**Scenario:** Your Jenkins pipeline deploys infrastructure to AWS using Terraform. Dev and Production require different instance sizes, and the EKS cluster name must be shown after deployment.

A clean configuration approach is:

1. Declare the inputs in variables.tf.
2. Set environment-specific values in dev.tfvars and prod.tfvars.
3. Validate permitted environment names and capacity limits.
4. Use locals.tf for common resource tags and naming expressions.
5. Use outputs.tf to expose useful details, such as cluster names and endpoints.
6. Retrieve secrets securely through Jenkins Credentials or an approved secrets manager.
7. Protect Terraform state and plan artifacts from unauthorized access.

> **ಕನ್ನಡ ವಿವರಣೆ:** Jenkins pipelineನಲ್ಲಿ ಯಾವ environment deploy ಮಾಡಬೇಕು ಅಂತ ಆಯ್ಕೆ ಮಾಡಬಹುದು. Devಗೆ ಒಂದು .tfvars file, Productionಗೆ ಇನ್ನೊಂದು file ಬಳಸಬಹುದು. Locals ಮೂಲಕ common tags maintain ಮಾಡಬಹುದು. Deployment ಆದ ಮೇಲೆ outputs ಮೂಲಕ cluster details ನೋಡಬಹುದು. Secrets ಅನ್ನು GitHubನಲ್ಲಿ ಇಡದೆ ಸುರಕ್ಷಿತವಾಗಿ manage ಮಾಡಬೇಕು.